CI/CD Workflows and End-to-End Architecture Guide
STM32Cube Preprocessing Project

Prepared for project operations and handover

July 30, 2026

Contents

1 Purpose **2**

2 Files in Scope **2**

3 Detailed Job Flow (Both Workflows) **2**
- 3.1 Job 1: Prepare matrix inputs . . . 2
- 3.2 Job 2: Check self-hosted runner availability . . . 2
- 3.3 Job 3: Preprocess <Series> (matrix) . . . 3
- 3.4 Job 4: Upload <Series> (matrix) . . . 3
- 3.5 Job 5: Runner unavailable (upload skipped) . . . 3

4 Differences Between the Two Workflow Files **4**
- 4.1 Auto Workflow: `auto_update_kb.yml` . . . 4
- 4.2 Manual Workflow: `manual_auto_update_kb.yml` . . . 4

5 Workflow Inputs and Defaults (If You Do Not Check/Set Anything) **4**
- 5.1 Manual Run (workflow_dispatch) Defaults . . . 4
- 5.2 Scheduled Auto Run Defaults . . . 4
- 5.3 Mode Interpretation . . . 5
- 5.4 Single Source of Truth for Series Governance . . . 5

6 End-to-End Architecture **5**
- 6.1 Control Plane (GitHub Actions) . . . 5
- 6.2 Execution Plane (Self-Hosted Runner in ST Network) . . . 5
- 6.3 Data and Artifacts . . . 6

7 Execution Scenarios **6**
- 7.1 Scenario A: Runner Available . . . 6
- 7.2 Scenario B: Runner Unavailable . . . 6

8 Operational Checklist **6**

9 Upload Timeout Policy **6**

10 Troubleshooting Guide **7**
- 10.1 Run shows only notice job . . . 7
- 10.2 Run button not visible . . . 7
- 10.3 Unsupported series error (for example C5) . . . 7
- 10.4 Upload did not happen . . . 7

11 Architecture Summary 7

1 Purpose

This document explains:

- The process implemented in `.github/workflows/auto_update_kb.yml`
- The process implemented in `.github/workflows/manual_auto_update_kb.yml`
- The complete execution architecture from trigger to upload

2 Files in Scope

File	Role
`.github/workflows/auto_update_kb.yml`	Main automation workflow. Supports scheduled and manual trigger. Contains default values for non-manual events and concurrency protection.
`.github/workflows/manual_auto_update_kb.yml`	Duplicate workflow. Triggered only by `workflow_dispatch`. Uses explicit user-provided inputs.
`pipeline_Automation/workflow_runner_single_series.py`	Called by both workflows for per-series processing and upload.

3 Detailed Job Flow (Both Workflows)

Both workflows use the same five-job architecture. The job names below match the workflow UI exactly.

3.1 Job 1: Prepare matrix inputs

Runs on: `ubuntu-latest`

Objective:

- Read raw workflow inputs
- Normalize series list
- Validate series dynamically from `shared/config/config_all_series.json`
- Export normalized outputs used by downstream jobs

Important behavior:

- If `series` is empty, all series from `all_series` are selected
- If `max_parallel` is empty, fallback is `1`
- Unsupported series immediately fail this job

3.2 Job 2: Check self-hosted runner availability

Runs on: `ubuntu-latest`

Objective:

- Query repository runners via GitHub API
- Compute availability metrics: `online_count`, `idle_count`, `busy_count`

- Produce gate output `runner_ready`

Availability rule:

- `runner_ready=true` only if at least one `self-hosted` runner is both `online` and `idle`

3.3 Job 3: Preprocess <Series> (matrix)

Job id: `preprocess-series`
Runs on: `windows-latest`
Timeout: `120` minutes

Condition:

- Runs only if `runner_ready == true`

What it does:

- Checks GitHub API rate-limit quota before ingestion
- Resolves Python executable and installs dependencies
- Calls `Run_Single_Series_Full_Pipeline_And_Upload.ps1` in `Prepare` mode
- Uploads produced artifacts as `st-ready-<series>-<run_number>`

3.4 Job 4: Upload <Series> (matrix)

Job id: `upload-series`
Runs on: `self-hosted`
Timeout: `90` minutes

Condition:

- Runs only if `mode != Prepare` and `runner_ready == true`

What it does:

- Downloads preprocess artifact if `mode == Full`
- Executes Alfred connectivity preflight (proxy/direct)
- Calls orchestration script in `Upload` mode with selected policies

3.5 Job 5: Runner unavailable (upload skipped)

Job id: `runner-unavailable-notice`
Runs on: `ubuntu-latest`

Condition:

- Runs only if `mode != Prepare` and `runner_ready != true`

What it does:

- Writes explicit metrics into Step Summary
- Fails intentionally (`exit 1`) to make skipped upload visible

This behavior prevents silent green runs when no self-hosted runner is available.

4 Differences Between the Two Workflow Files

4.1 Auto Workflow: `auto_update_kb.yml`

- Triggers: `schedule`, `workflow_dispatch`
- Has defaults for non-manual events via environment variables
- Uses concurrency group:
  `auto-update-kb-${{ github.ref }}`

4.2 Manual Workflow: `manual_auto_update_kb.yml`

- Trigger: `workflow_dispatch` only
- No schedule-based auto-execution
- Best for controlled on-demand execution by operator

5 Workflow Inputs and Defaults (If You Do Not Check/Set Anything)

5.1 Manual Run (workflow_dispatch) Defaults

Input	Type	Default	Effective behavior if untouched
`series`	text	empty	Empty means all allowed series from `shared/config/config_all_series.json`.
`mode`	choice	`Full`	Run preprocess and upload.
`existing_datasource_mode`	choice	`Replace`	Existing datasource content is refreshed.
`skip_drivers`	boolean	`false`	Unchecked means driver/subrepo uploads are included.
`skip_schema_validation`	boolean	`false`	Unchecked means schema validation gate is enforced.
`continue_on_workflow_error`	boolean	`false`	Unchecked means fail fast on workflow error.
`placeholder_policy`	choice	`Fail`	Unresolved Alfred placeholders fail the run.
`enable_issues_by_component_flow`	boolean	`false`	Unchecked means no component split/upload flow.
`component_issues_only`	boolean	`false`	Unchecked means monolithic issues datasource remains enabled.
`max_parallel`	text(number)	`1`	One series job at a time.

5.2 Scheduled Auto Run Defaults

For scheduled runs (non-manual event), `auto_update_kb.yml` does not read UI checkboxes. It uses internal defaults:

- `series=''` → all series
- `mode='Full'`
- `existing_datasource_mode='Replace'`
- `skip_drivers=false`

- `skip_schema_validation=false`
- `continue_on_workflow_error=false`
- `placeholder_policy='Fail'`
- `enable_issues_by_component_flow=false`
- `component_issues_only=false`
- `max_parallel='1'`

5.3 Mode Interpretation

- `Full`: preprocess + upload
- `Prepare`: preprocess only (upload job is skipped by condition)
- `Upload`: upload only (preprocess command is skipped)

5.4 Single Source of Truth for Series Governance

The two workflows and the per-series orchestration script now share one source of truth: `shared/config/config.`

- If `series` input is empty, workflows process all configured series from `all_series`
- If `series` is provided, each code is validated against this dynamic set
- Component scripts also validate from the same config (no static `ValidateSet`)

Operational consequence: adding a new series requires updating central config (and its per-series config file), not editing multiple workflow/script allowlists.

6 End-to-End Architecture

6.1 Control Plane (GitHub Actions)

1. Event trigger (schedule/manual)
2. Input preparation and validation
3. Self-hosted runner health gate
4. Conditional execution or explicit fail-fast notice

6.2 Execution Plane (Self-Hosted Runner in ST Network)

1. Source checkout and Python environment setup
2. Pipeline execution by series: ingestion → cleaning → enrichment → similarity → chunking → delivery
3. ST-ready export generation
4. Upload to Knowledge Base via ST-internal endpoint

6.3 Data and Artifacts

- Input/Intermediate datasets: `datasets/01_raw` to `datasets/07_delivery`
- KB-ready exports: `datasets/07_delivery/st_ready/by_series/`
- Optional artifact retention in GitHub Actions for traceability

7 Execution Scenarios

7.1 Scenario A: Runner Available

- Prepare and runner-check succeed
- Per-series processing starts
- Upload is executed
- Run finishes according to script success/failure

7.2 Scenario B: Runner Unavailable

- Prepare succeeds
- Runner check reports zero online+idle runner
- Pipeline job is skipped
- Notice job fails intentionally to signal no processing occurred

8 Operational Checklist

Before running either workflow:

1. Confirm self-hosted runner is `Online` in repository settings
2. Ensure runner is not busy
3. Verify runner environment variables are configured (not workflow secrets): `ST_GITHUB_ANALYZER_API_KEY` (or `PERSONA_API_KEY` / `ST_CHATGPT_API_KEY`)
4. Optional: define `ST_REMOTE_USER`; otherwise the orchestration script default is used
5. For manual runs, choose correct series and mode

9 Upload Timeout Policy

To avoid long blocking waits and unnecessary GitHub minutes consumption, each upload HTTP call is now bounded to **60 seconds maximum**.

- Implemented in uploader default: `Add_Data_Source_Files.py` (`-timeout`=60 by default)
- Enforced by orchestration override in full pipeline script (60s)
- Applied in component upload flow as explicit `-timeout 60`

10 Troubleshooting Guide

10.1 Run shows only notice job

Cause: no online+idle self-hosted runner.
Action: start runner process/service on the bound machine and rerun.

10.2 Run button not visible

Cause: workflow file missing or invalid on default branch.
Action: ensure valid `workflow_dispatch` YAML is present on default branch.

10.3 Unsupported series error (for example C5)

Cause: the series is not declared in `shared/config/config_all_series.json` or has no mapped per-series config file.
Action: add `STM32Cube<Series>` to `all_series`, map it in `series_config_files`, and create the corresponding `shared/config/config_<series>.json`.

10.4 Upload did not happen

Cause: pipeline stage did not run (runner gate blocked) or upload stage failed.
Action: inspect `run-series` logs and PowerShell script output.

11 Architecture Summary

The architecture is intentionally split into:

- **Cloud orchestration** (GitHub Actions): trigger, validation, gating, reporting
- **ST-network execution** (self-hosted runner): preprocessing and KB upload

This split is required because upload endpoints are not reachable from standard GitHub-hosted runners.